Avatar Generation Service — v3 Development Plan

Kokokino Spoke App · Mac Pro Dev · Galaxy + RunPod Deploy

Updated March 15, 2026

Executive Summary

Build a Kokokino spoke app that generates TalkingHead-ready 3D avatars from text prompts or images — with split lips, teeth, tongue, eyes, 52 ARKit blend shapes, 15 Oculus visemes, Mixamo-compatible rigging, and animations — all automatically.

Key architectural decisions in this revision:

- **Development machine**: 2013 Mac Pro (no GPU compute locally)
- **Blender**: v3.6 locally for light scripting/testing; v5.0 in cloud containers for production
- **GPU compute**: RunPod Serverless (pay-per-second, scales to zero)
- **Web hosting**: Meteor Galaxy (your existing deployment platform)
- **App framework**: Kokokino spoke_app_skeleton (Meteor + Mithril.js + SSO)
- **3D viewer**: Babylon.js (replaces Three.js from previous plan)
- **Image generation**: HiDream-I1 (MIT license, primary) + optional FLUX Kontext
- **3D generation**: TRELLIS.2 (MIT) + Hunyuan3D
- **Rigging**: UniRig (MIT)

---

1. Your Development Environment

2013 Mac Pro — What It Can and Can't Do

Can do (runs locally):

- Code editing, Claude Code, Git, SSH, Docker image building
- Meteor app development and local testing
- Blender 3.6 for basic script prototyping and template inspection
- Python scripting (deformation transfer math, prompt decomposition, API logic)
- Babylon.js frontend development in browser
- Docker image building (for pushing to RunPod)

Cannot do (must run in cloud):

- Blender 4.x or 5.x (requires macOS 13.0+; your Mac Pro tops out around macOS 12 Monterey or lower)
- Any ML model inference (HiDream, TRELLIS.2, UniRig — all need CUDA GPUs)
- GPU-accelerated rendering

Key implication: Blender 5.0 runs in Docker containers on RunPod for all production work. You develop Blender Python scripts locally on 3.6 for rapid iteration on logic, but always validate on 5.0 in the cloud before shipping. The bpy API is largely compatible between 3.6 and 5.0 for mesh operations, shape keys, and GLTF export — the main differences are in rendering, geometry nodes, and color management, which we don't rely on heavily.

Blender Version Strategy

Context	Version	Why
Local Mac Pro prototyping	3.6.x	Last version supporting older macOS. Use for script logic, template inspection, basic mesh edits
RunPod pipeline containers	5.0 (stable)	Current production stable. Full GLTF/GLB morph target export, latest bpy API
Compatibility target	4.2 LTS as minimum	Supported until July 2026. Scripts should work on 4.2+ for broadest compatibility

Blender 5.0 (released November 18, 2025) is the current stable release. **Blender 5.1** is in Release Candidate as of March 17, 2026 and will release imminently. Our Docker containers will pin to 5.0 initially and upgrade to 5.1 after it stabilizes.

Script compatibility approach: Write all Blender Python scripts targeting the bpy API common to 3.6–5.0. Test logic on 3.6 locally, validate on 5.0 via RunPod. The core operations we need (mesh manipulation, shape keys, armature/skinning, GLTF export) have stable APIs across these versions. Avoid 5.0-only features (ACES color management, new geometry nodes, SDF nodes) in the pipeline scripts.

---

2. Three-Tier Architecture

```
┌──────────────────────────────────────────────────────────────┐
│  YOUR 2013 MAC PRO (Development Only)                        │
│  Code editor + Claude Code + Git + Docker build              │
```

```
│  Blender 3.6 for script prototyping                                  │
│  SSH into RunPod GPU Pods for interactive GPU testing                 │
│  meteor deploy to Galaxy                                             │
│  docker push to Docker Hub → RunPod Serverless                       │
└──────────────────────────────────────────────────────────────────────┘

        │ deploy                          │ docker push
        ▼                                 ▼

┌──────────────────────┐   HTTP API   ┌──────────────────────────┐
│  METEOR GALAXY       │ ◀──────────▶ │  RUNPOD SERVERLESS GPU   │
│  Kokokino Spoke App  │              │  (scales to zero)        │
│                      │              │                          │
│  • Meteor frontend   │  webhook     │  Endpoint 1: HiDream-I1  │
│    (Mithril.js)      │ ◀──────────  │   Image gen, RTX 4090    │
│  • Babylon.js viewer │              │   ~$0.69/hr              │
│  • Node.js API       │              │                          │
│  • MongoDB           │              │  Endpoint 2: TRELLIS.2   │
│  • Job queue/status  │              │   3D mesh, RTX 5090      │
│  • SSO via Hub       │              │   ~$0.89/hr              │
│  • GLB file serving  │              │                          │
│  • User gallery      │              │  Endpoint 3: UniRig      │
│                      │              │   Auto-rig, RTX 4090     │
│  Cost: $0-30/mo      │              │   ~$0.69/hr              │
│                      │              │                          │
│                      │              │  Endpoint 4: Blender 5.0 │
│                      │              │   Headless processing    │
│                      │              │   CPU or GPU, ~$0.20/hr  │
│                      │              │                          │
│                      │              │  Endpoint 5: HiDream-E1.1 │
│                      │              │   Image editing, RTX 4090 │
│                      │              │   ~$0.69/hr              │
│                      │              │                          │
│                      │              │  Shared: Network Storage │
│                      │              │   Template meshes, ICT   │
│                      │              │   FaceKit, Mixamo anims  │
│                      │              │                          │
│                      │              │  Cost: $0 idle,          │
│                      │              │   ~$0.15-0.50/avatar     │
└──────────────────────┘              └──────────────────────────┘

        │
        ▼

┌──────────────────────────────────────────────────────────────────────┐
│  USER'S BROWSER                                                      │
│  Babylon.js loads GLB from Galaxy CDN                                │
│  TalkingHead-compatible lip-sync + animations                        │
```

```
│  HeadTTS / HeadAudio for viseme-driven speech                    │
└──────────────────────────────────────────────────────────────────┘
```

Why RunPod Over DigitalOcean

DigitalOcean offers GPU Droplets but they're always-on VMs — you'd pay whether anyone is generating avatars or not. RunPod Serverless scales to zero workers when idle, charges per-second only during active generation, and offers a wider GPU selection (RTX 4090 at ~$0.69/hr, RTX 5090 at ~$0.89/hr, A100 at ~$1.74/hr). The serverless model means your idle cost is $0 for GPU compute. RunPod also has pre-built Docker templates for AI models and ComfyUI, network storage shared across endpoints, and 30+ global data centers.

For your use case — bursty GPU demand (someone generates an avatar, then nothing for a while) — serverless is dramatically more cost-effective than always-on VMs.

---

3. Kokokino Spoke App Integration

Base Template

Fork from `kokokino/spoke_app_skeleton` which provides:

- Meteor application structure (client/, server/, imports/)
- SSO integration with Kokokino Hub (token validation, session management)
- Subscription checking middleware (gate avatar generation behind paid plans)
- MongoDB collections for data storage
- Mithril.js for lightweight reactive UI components
- CLAUDE.md for Claude Code guidance
- rspack.config.js for bundling
- MIT license

What We Add to the Spoke App

```
spoke_app_skeleton/
├── .meteor/
├── client/
|   ├── main.js                   # Existing
|   └── views/
|       ├── AvatarGenerator.js     # NEW: Main generation UI (Mithril component)
|       ├── AvatarPreview.js       # NEW: Babylon.js 3D preview (Mithril
component)
```

```
│       ├── AvatarGallery.js       # NEW: User's saved avatars
│       ├── AvatarCustomizer.js    # NEW: Body/face sliders
│       └── TalkingHeadDemo.js     # NEW: Live lip-sync test
├── imports/
│   ├── api/
│   │   ├── avatars/               # NEW: Avatar collection + methods
│   │   │   ├── collection.js
│   │   │   ├── methods.js         # generate, refine, customize, download
│   │   │   └── publications.js
│   │   └── jobs/                  # NEW: Job queue collection
│   │       ├── collection.js
│   │       ├── methods.js         # createJob, cancelJob, getStatus
│   │       └── publications.js
│   ├── lib/
│   │   ├── runpod-client.js       # NEW: RunPod API integration
│   │   ├── babylon-loader.js      # NEW: Babylon.js GLB loading + morph targets
│   │   └── prompt-decomposer.js   # NEW: LLM prompt parsing
│   └── ui/                        # Existing Mithril components
├── server/
│   ├── main.js                    # Existing
│   ├── runpod-webhooks.js         # NEW: Receive RunPod job completion
│   └── avatar-pipeline.js         # NEW: Orchestrate multi-step generation
├── public/
│   └── avatars/                   # NEW: Served GLB files (or S3/CDN)
├── CLAUDE.md                      # Existing (update with avatar-specific
guidance)
├── package.json                   # Add babylonjs, runpod SDK
└── settings.json                  # Add RunPod API keys, model configs
```

Babylon.js Instead of Three.js

Babylon.js fully supports GLB loading with morph targets — the GLTF 2.0 loader reads morph target names from the `extras.targetNames` array and creates `MorphTarget` objects accessible via `mesh.morphTargetManager`. You can set individual morph target influence values (0.0–1.0) programmatically, which is exactly what TalkingHead does for lip-sync.

Key Babylon.js integration points:

```javascript
```

```
// Loading a GLB with morph targets
import { SceneLoader } from '@babylonjs/core/Loading/sceneLoader';
import '@babylonjs/loaders/glTF';

const result = await SceneLoader.ImportMeshAsync('', '/avatars/', 'avatar.glb', scen
const headMesh = result.meshes.find(m => m.morphTargetManager);

// Setting a viseme (same concept as TalkingHead but in Babylon.js)
const mtm = headMesh.morphTargetManager;
for (let i = 0; i < mtm.numTargets; i++) {
  const target = mtm.getTarget(i);
  if (target.name === 'viseme_aa') {
    target.influence = 0.8; // Open mouth for "aa" sound
  }
}

// Animating morph targets for lip-sync
const animation = new BABYLON.Animation('lipSync', 'influence', 30,
  BABYLON.Animation.ANIMATIONTYPE_FLOAT, BABYLON.Animation.ANIMATIONLOOPMODE_CYCLE);
```

TalkingHead compatibility note: TalkingHead itself is built on Three.js. For your Babylon.js spoke app, you have two approaches:

1. **Reimplement the lip-sync logic in Babylon.js** — HeadAudio outputs Oculus viseme blend shape values as numbers; HeadTTS outputs phoneme timestamps. Your Babylon.js code reads these values and sets `morphTarget.influence` accordingly. The actual lip-sync math is straightforward — it's just setting morph target weights over time.
2. **Embed TalkingHead in an iframe** for the lip-sync demo page while using native Babylon.js for the main 3D preview/customization. This is faster to build initially.

I'd recommend approach 1 for production (native Babylon.js) but approach 2 for MVP speed.

---

4. RunPod Serverless Endpoints — Detailed Setup

Each pipeline stage is a separate Docker container deployed as a RunPod Serverless endpoint. You build these containers on your Mac Pro (Docker for Mac) and push to Docker Hub.

Endpoint 1: HiDream-I1 (Image Generation)

```dockerfile
```

```
FROM pytorch/pytorch:2.4.0-cuda12.4-cudnn9-runtime
RUN pip install diffusers transformers accelerate runpod
# Download model weights at build time (baked into image)
RUN python -c "from diffusers import HiDreamImagePipeline; \
    HiDreamImagePipeline.from_pretrained('HiDream-ai/HiDream-I1-Dev')"
COPY handler.py /app/handler.py
CMD ["python", "/app/handler.py"]
```

```python
# handler.py
import runpod
from diffusers import HiDreamImagePipeline
import torch, base64, io
from PIL import Image

pipe = HiDreamImagePipeline.from_pretrained(
    "HiDream-ai/HiDream-I1-Dev", torch_dtype=torch.bfloat16).to("cuda")

def handler(event):
    prompt = event["input"]["prompt"]
    image = pipe(prompt, height=1024, width=1024,
                 num_inference_steps=28, guidance_scale=0.0).images[0]
    buffer = io.BytesIO()
    image.save(buffer, format="PNG")
    return {"image_base64": base64.b64encode(buffer.getvalue()).decode()}

runpod.serverless.start({"handler": handler})
```

GPU: RTX 4090 (24GB) — HiDream-I1-Dev fits comfortably **Cost**: ~$0.69/hr → ~$0.02–0.04 per image (7–15 seconds generation)

Endpoint 2: TRELLIS.2 (3D Mesh Generation)

GPU: RTX 5090 (32GB) recommended, or A100 (80GB) for maximum quality **Cost**: ~$0.89/hr (5090) → ~$0.05–0.25 per mesh (17–60 seconds)

Endpoint 3: UniRig (Auto-Rigging)

GPU: RTX 4090 (24GB) — UniRig is relatively lightweight **Cost**: ~$0.69/hr → ~$0.01–0.02 per rig (1–5 seconds)

Endpoint 4: Blender 5.0 Headless (Mesh Processing)

This is the workhorse endpoint that handles:

- Template mesh parametric variation
- Retopology cleanup
- Face component separation
- Deformation transfer (ARKit blend shapes)
- Oculus viseme generation
- Texture projection/baking
- Final GLB export with all morph targets

dockerfile

```dockerfile
FROM ubuntu:24.04
# Install Blender 5.0 headless
RUN apt-get update && apt-get install -y wget xz-utils libx11-6 libgl1
RUN wget https://download.blender.org/release/Blender5.0/blender-5.0-linux-x64.tar.x
    && tar xf blender-5.0-linux-x64.tar.xz -C /opt/ \
    && ln -s /opt/blender-5.0-linux-x64/blender /usr/local/bin/blender
# Install Python dependencies into Blender's Python
RUN /opt/blender-5.0-linux-x64/5.0/python/bin/python3 -m pip install \
    numpy scipy runpod
# Copy pipeline scripts
COPY blender_scripts/ /app/blender_scripts/
COPY templates/ /app/templates/
COPY handler.py /app/handler.py
CMD ["python3", "/app/handler.py"]
```

python

```python
# handler.py
import runpod, subprocess, json, os

def handler(event):
    task = event["input"]["task"]  # "parametric", "blendshapes", "export", etc.
    params = event["input"]["params"]

    # Write params to temp file for Blender script to read
    with open("/tmp/params.json", "w") as f:
        json.dump(params, f)

    # Run Blender headless with appropriate script
    script_map = {
        "parametric": "/app/blender_scripts/template_parametric.py",
        "blendshapes": "/app/blender_scripts/generate_arkit_shapes.py",
        "visemes": "/app/blender_scripts/generate_oculus_visemes.py",
        "face_separation": "/app/blender_scripts/separate_face_parts.py",
        "texture_project": "/app/blender_scripts/project_textures.py",
        "export_glb": "/app/blender_scripts/export_talkinghead_glb.py",
    }

    result = subprocess.run([
        "blender", "--background", "--python", script_map[task],
        "--", "/tmp/params.json"
    ], capture_output=True, text=True, timeout=300)

    # Return path to output file (on RunPod network storage)
    return {"output_path": f"/runpod-vol/outputs/{params['job_id']}.glb",
            "stdout": result.stdout, "stderr": result.stderr}

runpod.serverless.start({"handler": handler})
```

GPU: CPU-only endpoint is fine for most Blender operations (~$0.20/hr). Use GPU instance only if doing Cycles rendering.

Shared Network Storage

RunPod network volumes persist across endpoint invocations. Store:

- Template meshes (.blend files)
- ICT FaceKit reference meshes
- Mixamo animation FBX library
- In-progress pipeline artifacts (intermediate meshes between endpoints)

- Final GLB outputs (before upload to Galaxy/CDN)

---

5. Pipeline Orchestration

The Meteor server orchestrates the multi-step pipeline by calling RunPod endpoints in sequence:

```javascript
```

```
// server/avatar-pipeline.js
import { RunPodClient } from '../imports/lib/runpod-client';

export async function generateAvatar(jobId, prompt, options) {
  const runpod = new RunPodClient(Meteor.settings.runpod.apiKey);

  // Step 1: Decompose prompt (runs on Galaxy server, no GPU needed)
  JobsCollection.update(jobId, { $set: { status: 'decomposing_prompt', progress: 5 }
  const attributes = await decomposePrompt(prompt);

  // Step 2: Generate hero image (RunPod: HiDream-I1)
  JobsCollection.update(jobId, { $set: { status: 'generating_concept', progress: 15
  const heroResult = await runpod.run('hidream-endpoint-id', {
    prompt: buildHiDreamPrompt(attributes),
  });
  // Save preview image, wait for user approval...

  // Step 3: Generate face close-up (RunPod: HiDream-I1)
  JobsCollection.update(jobId, { $set: { status: 'generating_face', progress: 30 }})
  const faceResult = await runpod.run('hidream-endpoint-id', {
    prompt: buildFacePrompt(attributes),
  });

  // Step 4: Select + parametrize template (RunPod: Blender 5.0)
  JobsCollection.update(jobId, { $set: { status: 'preparing_mesh', progress: 45 }});
  const meshResult = await runpod.run('blender-endpoint-id', {
    task: 'parametric',
    params: { template: attributes.template, body: attributes.body_params,
              face: attributes.face_params, job_id: jobId }
  });

  // Step 5: Project textures (RunPod: Blender 5.0)
  JobsCollection.update(jobId, { $set: { status: 'texturing', progress: 55 }});
  await runpod.run('blender-endpoint-id', {
    task: 'texture_project',
    params: { mesh_path: meshResult.output_path, face_image: faceResult.image_base64
              body_image: heroResult.image_base64, job_id: jobId }
  });

  // Step 6: Generate blend shapes (RunPod: Blender 5.0)
  // (Templates already have these, but run validation)
  JobsCollection.update(jobId, { $set: { status: 'blend_shapes', progress: 70 }});
  await runpod.run('blender-endpoint-id', {
```

```
    task: 'blendshapes',
    params: { mesh_path: meshResult.output_path, job_id: jobId }
  });

  // Step 7: Export final GLB (RunPod: Blender 5.0)
  JobsCollection.update(jobId, { $set: { status: 'exporting', progress: 85 }});
  const exportResult = await runpod.run('blender-endpoint-id', {
    task: 'export_glb',
    params: { job_id: jobId }
  });

  // Step 8: Upload GLB to Galaxy/CDN
  JobsCollection.update(jobId, { $set: { status: 'uploading', progress: 95 }});
  const glbUrl = await uploadToStorage(exportResult.output_path);

  // Done!
  JobsCollection.update(jobId, {
    $set: { status: 'complete', progress: 100, glb_url: glbUrl }
  });
  AvatarsCollection.insert({
    userId: job.userId, glbUrl, prompt, attributes, createdAt: new Date()
  });
}
```

Real-time progress updates reach the user's browser via Meteor's DDP reactive publications — the JobsCollection is published and the Mithril UI reactively updates the progress bar.

---

6. Claude Code Development Phases

Phase 0: Foundation (1–2 weeks)

On Mac Pro (local):

```
- Fork kokokino/spoke_app_skeleton
- Set up Meteor development environment
- Install Babylon.js: npm install @babylonjs/core @babylonjs/loaders
- Create basic AvatarGenerator Mithril component with prompt input
- Create basic AvatarPreview component with Babylon.js GLB viewer
  (test with a sample RPM-compatible GLB)
- Set up RunPod API client module (HTTP calls to RunPod endpoints)
- Set up JobsCollection + publications for real-time progress
- Write prompt decomposition logic (Claude API call or rule-based)
- Test Blender 3.6 Python scripts for:
  - Loading a template .blend file
  - Applying shape key parameters
  - Listing morph targets
  - Exporting as GLB
  - Verify these scripts also work on Blender 5.0 (test via RunPod Pod)
```

On RunPod GPU Pod (SSH from Mac, ~$5-10 spend):

```
- Spin up RTX 4090 Pod
- Install HiDream-I1-Dev, test generation
- Install TRELLIS.2, test image-to-3D
- Install UniRig, test skeleton generation
- Install Blender 5.0, verify Mac-written scripts run correctly
- Benchmark VRAM usage and generation time for each model
- Tear down Pod when done
```

Deliverable: Working spoke app skeleton with Babylon.js viewer and RunPod API client. Each ML component verified on cloud GPU.

Phase 1: Template Mesh System (2–3 weeks)

On Mac Pro (Blender 3.6):

```
- Create 3-5 base body templates in Blender 3.6:
  (These are .blend files with mesh, rig, UVs — no 5.0 features needed)
  - Male realistic, female realistic, androgynous
  - Stylized/cartoon, anime-style
  - Each with separate objects: body, eyes, teeth, tongue, eyelashes
  - Each with Mixamo-compatible skeleton
  - Each UV-unwrapped
  - Target: 8k-10k faces per template

- Write parametric variation scripts (Blender Python):
  - template_parametric.py: body shape keys + face shape keys
  - Test in Blender 3.6 locally

- Write deformation transfer scripts (pure Python/NumPy):
  - generate_arkit_shapes.py: transfer 52 ARKit shapes from ICT FaceKit
  - generate_oculus_visemes.py: derive 15 Oculus visemes
  - These are math-heavy, not Blender-specific — run on Mac CPU

- Write GLB export script:
  - export_talkinghead_glb.py: export with all morph targets, correct naming
  - Test locally in 3.6, validate on 5.0 via RunPod
```

On RunPod (validate + build Docker images):

```
- Upload templates to RunPod network storage
- Run all Blender scripts on Blender 5.0 to verify compatibility
- Build Blender 5.0 Docker container with scripts + templates baked in
- Push to Docker Hub
- Deploy as RunPod Serverless endpoint
- Test end-to-end: API call → parametric mesh → GLB output
```

Deliverable: Template GLBs that work in Babylon.js viewer and TalkingHead.

Phase 2: AI Appearance Pipeline (2-3 weeks)

On Mac Pro:

```
- Write HiDream prompt engineering module
  - Build prompts optimized for T-pose character generation
  - Route structured attributes to HiDream's four text encoders
  - Test via RunPod API calls from Mac

- Write texture projection Blender script
  - Project face close-up onto template UV island
  - Test in Blender 3.6 locally

- Build approval flow in Meteor app
  - Show generated concept images
  - User picks favorite or requests regeneration
  - Refinement via HiDream-E1.1 ("make hair longer")

- Integrate Babylon.js preview
  - Load textured GLB in real-time
  - Morph target slider controls for testing blend shapes
```

On RunPod:

```
- Build HiDream-I1 Docker container, deploy as serverless endpoint
- Build HiDream-E1.1 Docker container, deploy as serverless endpoint
- Test full flow: prompt → image → texture → GLB
```

Phase 3: AI Mesh Path (2–3 weeks)

On Mac Pro:

```
- Write retopology pipeline script (calls Instant Meshes + Blender cleanup)
- Write face separation script (MediaPipe landmarks → vertex selection)
- Write quality scoring module
- Integrate into Meteor pipeline orchestrator as alternative path
```

On RunPod:

```
- Build TRELLIS.2 Docker container, deploy as serverless endpoint
- Build UniRig Docker container, deploy as serverless endpoint
- Test full AI mesh path end-to-end
- Validate generated GLBs in Babylon.js + TalkingHead
```

Phase 4: Animation + Polish (1–2 weeks)

On Mac Pro:

```
- Download Mixamo animation FBX library
- Write animation retargeting Blender script
- Write validation suite:
  - All 72 morph targets present and correctly named
  - Skeleton bone count matches Mixamo convention
  - Mesh face count within target range
  - No inverted normals, manifold check
- Polish Meteor UI: gallery, download options, sharing
```

On RunPod:

```
- Upload Mixamo animations to network storage
- Integrate animation attachment into Blender endpoint
- Final end-to-end testing
- Deploy to production Galaxy + RunPod
```

Phase 5: Production Deploy (1–2 weeks)

```
- Deploy Meteor spoke app to Galaxy (meteor deploy)
- Configure Kokokino Hub SSO for production domain
- Set up subscription tiers:
  - Free: 3 avatars/month (template path only)
  - Pro: 50 avatars/month (template + AI mesh path)
  - Studio: unlimited + API access
- Set up S3/CDN for GLB file serving
- Monitoring: RunPod dashboard + Galaxy APM
- Error handling: automatic fallback from AI path to template path
- Launch!
```

---

7. Cost Analysis

Development Phase

Item	Cost
Mac Pro (existing)	$0

Item	Cost
Galaxy free/sandbox plan	$0
RunPod GPU Pods for testing (~20 hours total)	~$15–30
RunPod Serverless testing (100 test avatars)	~$15–50
Docker Hub (free tier)	$0
MongoDB Atlas (free tier or Galaxy managed)	$0
Total development cost	**~$30–80**

Production (Monthly)

Scale	Galaxy	RunPod	Storage	Total
Hobby (5 avatars/day)	$0–10	~$5–15	$5	~$10–30/mo
Small (50/day)	$30	~$50–150	$20	~$100–200/mo
Medium (500/day)	$100	~$500–1500	$50	~$650–1650/mo

Per-Avatar Cost Breakdown

Step	GPU Time	Cost
HiDream hero image	~10s on RTX 4090	$0.002
HiDream face close-up	~10s on RTX 4090	$0.002
Blender parametric + texture	~30s on CPU	$0.002
Blender blend shapes + export	~60s on CPU	$0.003
Template path total	~2 min	**~$0.01**
+ TRELLIS.2 mesh (AI path)	~20s on RTX 5090	$0.005
+ UniRig rigging (AI path)	~5s on RTX 4090	$0.001
+ Blender retopo/separation	~120s on CPU	$0.007
AI mesh path total	~4 min	**~$0.02**

These are remarkably low per-unit costs. Even with overhead (RunPod cold starts, network transfer, storage), each avatar should cost well under $0.10 in compute. This allows generous free tiers and healthy margins on paid plans.

---

8. Technology Stack Summary

Core Stack (All MIT/Apache/BSD — Commercially Clean)

Component	License	Role
Meteor 3.x	MIT	App framework (spoke app)
Mithril.js	MIT	UI components (from spoke skeleton)
Babylon.js	Apache 2.0	3D viewer, morph target animation
HiDream-I1	MIT	Concept image generation
HiDream-E1.1	MIT	Image editing/refinement
TRELLIS.2	MIT	Image → 3D mesh
UniRig	MIT	Auto skeleton + skinning
Blender 5.0	GPL	Headless mesh processing
ICT FaceKit	Free/Academic	ARKit blend shape reference
Deformation Transfer	Open	Blend shape generation
Instant Meshes	BSD	Automatic retopology
MediaPipe	Apache 2.0	Face landmark detection
rembg	MIT	Background removal
RunPod SDK	MIT	Serverless GPU orchestration
MongoDB	SSPL	Database (via Galaxy managed)

Infrastructure

Service	Role	Cost Model
Meteor Galaxy	Web app hosting	Per-container/minute
RunPod Serverless	GPU compute	Per-second, scales to zero
Docker Hub	Container registry	Free tier
CloudFlare R2 or S3	GLB file CDN	Per-GB stored + transferred
Kokokino Hub	SSO + billing	Part of Kokokino ecosystem

---

9. Timeline Summary

Phase	Duration	What Runs Where
Phase 0: Foundation	1–2 weeks	Mac: Meteor app + Babylon.js. RunPod Pod: ML smoke tests
Phase 1: Templates	2–3 weeks	Mac: Blender 3.6 modeling + scripts. RunPod: validate on 5.0
Phase 2: AI Appearance	2–3 weeks	Mac: prompt engineering + UI. RunPod: HiDream endpoints
Phase 3: AI Mesh Path	2–3 weeks	Mac: pipeline logic. RunPod: TRELLIS.2 + UniRig endpoints
Phase 4: Animation + Polish	1–2 weeks	Mac: UI polish. RunPod: final integration
Phase 5: Production Deploy	1–2 weeks	Galaxy: deploy app. RunPod: production endpoints
Template MVP	**~8 weeks**	
Full service	**~14 weeks**	

---

10. Key Risks & Mitigations

Risk	Mitigation
Blender 3.6 → 5.0 script incompatibility	Write to common API subset. Test on 5.0 via RunPod early and often

Risk	Mitigation
RunPod cold start latency (first request after idle)	Use FlashBoot (sub-200ms for pre-warmed). Keep 1 warm worker on busy endpoints
Babylon.js morph target naming mismatch	Validate GLB morph target names match GLTF spec. Babylon reads `targetNames` from GLTF extras
HiDream generates inconsistent T-poses	Fine-tune prompt engineering. Add ControlNet/pose conditioning if available
Template path feels too "samey"	Invest in parametric variation system. Many shape key combinations = visual diversity
Face separation fails on AI meshes	Auto-fallback to template path. Score quality, reject if below threshold
Galaxy can't serve large GLB files efficiently	Use CloudFlare R2 or S3 for GLB storage. Galaxy serves the app, CDN serves the files
Kokokino Hub SSO integration issues	Spoke skeleton already handles this. Follow existing patterns